

Business Requirements Document

CheatMealFinder

Local Street Food & Food Truck Discovery Platform

A database-driven solution for finding signature dishes and honest street food reviews

Document Version	1.0	Status	DRAFT FOR REVIEW
Date	24 September 2026	Classification	Internal
Author	Project Team (Student Developers)	Stakeholder	Faculty Advisor
Review Cycle	Weekly code & design reviews	Next Review	Week 4

Purpose of This Document

This BRD defines **what** the CheatMealFinder platform must do and **why** it is being built, before any code is written. It is the reference for scope decisions, feature acceptance, and stakeholder sign-off. Any new feature not listed here requires a formal change request.

1. Executive Summary

CheatMealFinder is a web platform that helps people find exactly the street food they are craving — from “the spiciest wings in town” to “the best late-night waffles” — while giving local food trucks and street vendors a digital presence they currently lack.

Today, discovering great street food depends entirely on word of mouth. Vendors have no central listing, and hungry customers have no reliable way to search by dish, spice level, price, or rating. CheatMealFinder solves this with a structured database of **vendors, dishes, categories, and user reviews**, searchable in seconds.

Beyond its practical value, the project is designed as a hands-on learning vehicle: the entire platform is built around core SQL concepts — **foreign keys** to link reviews to dishes and vendors, **JOINS** to assemble detail pages, and **GROUP BY** aggregates to rank the highest-rated food trucks. Success is measured in user engagement, query performance, and demonstrated database integrity — not revenue.

2. Business Objectives (SMART)

Each objective below is Specific, Measurable, Achievable, Relevant, and Time-bound.

#	Objective	How We Measure It	Target	Deadline
O1	Build a complete vendor & dish database	Number of active records	100 vendors · 300 dishes	End of Semester 1
O2	Achieve real user engagement	Registered users · total reviews	200 users · 500 reviews	Week 8 after launch
O3	Deliver fast search	Query response time	Under 2 seconds	At launch
O4	Keep review quality high	% of reviews flagged as spam	Less than 3%	Week 8 after launch
O5	Guarantee data integrity	Database audit	0 orphaned records	Before launch sign-off

3. Scope Definition

Scope fixes the boundary of the project. Everything in the first list will be delivered; everything in the second list is explicitly excluded to prevent scope creep.

■ IN SCOPE — What We Will Build

- Vendor database: street food vendors & food trucks (name, location, opening hours, contact details)
- Dish database: each vendor's signature dishes with price, category, spice level, and description
- Categories: Street Food, Dessert, Indo-Chinese — designed to be extended later
- User accounts: registration with email and password; one account per email address
- Reviews: 1–5 star ratings with written comments, stored against the dish being reviewed
- Smart search: find dishes by keyword, category, price range, and spice level
- Rankings: 'Top Rated Food Trucks' leaderboard generated with GROUP BY average-rating queries

- Admin panel: manage vendors, dishes, and moderate flagged reviews

■ OUT OF SCOPE — Explicitly Excluded (v2.0 Candidates)

- Online ordering, payments, or escrow
- Live GPS tracking of food trucks
- Delivery integration with third-party services
- Vendor self-service dashboards
- Native iOS/Android mobile apps (the MVP is web-only)
- AI-powered recommendation engine

4. Stakeholder Analysis

Understanding who is affected by the platform ensures requirements reflect real needs.

Stakeholder	Their Problem Today	What They Need From Us	Influence
Users (Food Lovers)	Cannot find a specific craving quickly; rely on scattered word of mouth	Keyword dish search and trustworthy ratings	High
Vendors & Food Trucks	Signature dishes invisible online; no free marketing channel	Free listing that reaches local customers	Medium
Project Team (Students)	Need demonstrable, real-world SQL and full-stack experience	A live project using FKs, JOINS, and GROUP BY	High
Faculty Advisor	Must verify that learning outcomes are met	Documented schema, queries, and weekly reviews	Critical
Platform Admin	Risk of spam and fake reviews damaging trust	Moderation tools and a review-flagging workflow	High

5. Functional Requirements

Each requirement has a unique ID, a priority (Must Have = launch blocker, Should Have = desirable), and testable acceptance criteria.

ID	Requirement	Priority	Acceptance Criteria (how we verify it works)
FR-01	Users register with email and password before they can review	Must Have	Duplicate emails are rejected; passwords are hashed, never stored in plain text
FR-02	Admin can add and edit vendors (name, location, hours, truck/stall type)	Must Have	Every vendor gets a unique ID; changes keep an edit history
FR-03	Dishes are added and linked to their vendor via a foreign key	Must Have	A dish cannot be saved without a valid vendor_id — the database blocks orphans
FR-04	Every dish has a category and a spice level from 1 to 5	Must Have	category_id is enforced as a foreign key; spice level restricted to 1–5

ID	Requirement	Priority	Acceptance Criteria (how we verify it works)
FR-05	Users search dishes by keyword, category, price, and spice level	Must Have	Combined filters work together and return results in under 2 seconds
FR-06	Users can rate and review a dish (1–5 stars plus written comment)	Must Have	Each review stores dish_id and user_id; a user can review each dish only once
FR-07	The site shows a 'Top Rated Food Trucks' leaderboard	Must Have	Ranked by average rating using GROUP BY; only vendors with 3+ reviews are ranked
FR-08	Users can run craving searches such as 'spiciest wings'	Must Have	Results match the keyword and are sorted by spice level, highest first
FR-09	Admin can flag or remove spam and abusive reviews	Should Have	Removal requires a logged reason; records are soft-deleted, not erased
FR-10	Each vendor has a detail page showing dishes, average rating, and reviews	Must Have	One JOIN query correctly assembles vendor + dishes + reviews

6. Non-Functional Requirements

These constraints define the quality standards the platform must meet — they apply to every feature.

Category	Requirement
Security & Privacy	Only name and email are stored; passwords hashed with bcrypt; prepared statements only (no SQL injection)
Performance	Search queries respond in under 2 seconds; pages load in under 3 seconds on normal Wi-Fi
Data Integrity	All relationships enforced by foreign keys at the database level; zero orphaned records permitted
Budget	Zero licensing cost — open-source stack only (MySQL/PostgreSQL + PHP, Python, or Node.js); free-tier hosting
Timeline	Minimum viable product (MVP) live before mid-semester exams, Week 10
Accessibility	Basic WCAG compliance: image alt text, readable colour contrast, keyboard-friendly forms

7. Database Design (Core Tables)

The platform is built around four linked tables. Arrows in the diagram below show the foreign-key relationships that keep the data consistent.

Table	Key Fields	What It Stores
Vendors	vendor_id (PK) · name · location · hours · vendor_type	Food trucks and street stalls
Categories	category_id (PK) · name	Street Food, Dessert, Indo-Chinese
Dishes	dish_id (PK) · vendor_id (FK) · category_id (FK) · name · price · spice_level · description	Each vendor's signature dishes
UserReviews	review_id (PK) · dish_id (FK) · user_id (FK) · rating · comment · created_at	User ratings and written reviews

Key SQL Concepts Demonstrated

Foreign keys link reviews → dishes → vendors so no review can exist without a real dish, and no dish without a real vendor. JOINS assemble vendor detail pages from multiple tables in one query. GROUP BY with AVG() produces the top-rated leaderboard, and ORDER BY with filters powers craving searches such as 'spiciest wings'.

8. Risk Assessment

Risks are ranked by impact and likelihood; each has a practical mitigation the team commits to.

Risk	Impact	Likelihood	Mitigation
Empty database at launch discourages first users	High	Medium	Seed the database with 30+ real local vendors before launch
Fake or spam reviews damage platform trust	High	Medium	Limit one review per user per dish; admin moderation with flag workflow
Duplicate or incorrect vendor entries	Medium	High	Unique constraint on name + location; admin approves every new vendor
Scope creep (ordering, payments, maps, AI)	Medium	High	Formal change-request process; BRD sign-off required for any new feature
Search queries slow down as data grows	Medium	Low	Index all search and filter columns; load-test with 10,000+ dummy records

9. Success Criteria & KPIs

Level	KPI	Target
Primary	Registered users and total reviews	200 users · 500 reviews by Week 8
Secondary	Search speed and content coverage	Queries under 2 seconds · 100+ vendors listed
Tertiary	Review quality (spam rate)	Under 3% of reviews flagged
Learning	Team SQL proficiency	Every member writes FK schema, JOINS, and GROUP BY queries unaided
Quality	Reliability at launch	Zero orphaned records · zero critical bugs in first 30 days

10. Assumptions & Dependencies

The plan below holds only if these conditions are true. Any change must be flagged immediately.

- Team members have basic SQL knowledge and have agreed on a web development stack
- The chosen hosting provider supports our database (MySQL or PostgreSQL) on a free tier
- Initial vendor data can be collected manually (walk-around survey, web research) for seeding
- A faculty advisor is available for weekly code and design reviews
- All team members have completed basic Git version-control training

11. Approval & Sign-Off

This document is approved as the baseline for the CheatMealFinder project. Changes after sign-off require a formal change request.

Role	Name	Signature	Date
Team Lead			
Faculty Advisor			
Stakeholder Representative			

Document history: v1.0 (24 Sep 2026) — Initial draft for review.